操作系统实验文档

Lab 12 - Synchronization 2

64d8bdae

Built by github-actions, at 2025-05-06 11:32:50+08:00

1. Weekly Lab Material

1.0.1 1.2 Week 12 - Synchronization 2

1.2.1 Synchronization 2

1.2.2 Experiment Objectives

1. Understand the principle of semaphores
2. Understand the producer-consumer model

DATA RACE

In the course content of Synchronization 1, we did not provide a precise definition of data race. Let's define it now:

Two (or more) memory operations **conflict** if they access the same location and at least one of them is a write operation.

If multiple memory accesses target the same address and at least one of them is a write, we call this situation a data race.

If we prevent writes, can we read concurrently?

This is the design principle of `rwlock`. We categorize access to shared resources (memory) into read and write operations. The `lock` operation is divided into `rdlock` and `wrlock`.

At any given moment, multiple readers can read concurrently (returning from `rdlock` simultaneously), or there can be only one writer. A single writer is mutually exclusive with all readers and other writers.

Built by github-actions, at 2025-05-06 11:32:50+08:00

If a data race has no side effects

If we allow data races but ensure they do not cause bugs, we can avoid using locks.

In computer science, read-copy-update (RCU) is a synchronization mechanism that avoids the use of lock primitives while multiple threads concurrently read and update elements that are linked through pointers and that belong to shared data structures (e.g., linked lists, trees, hash tables).

Whenever a thread is inserting or deleting elements of data structures in shared memory, all readers are guaranteed to see and traverse either the older or the new structure, therefore avoiding inconsistencies (e.g., dereferencing null pointers).

See also: https://www.kernel.org/doc/html/next/RCU/whatisRCU.html

SEMAPHORE

A semaphore is a **mutex with a counter**.

We can understand a semaphore as a "finite number of shared resources," such as lockers in a swimming facility or parking spaces in a parking lot.

A swimming facility has a limited number of wristbands. Upon entering, each person receives a wristband (holding the semaphore); only those with wristbands can enter (mutual exclusion). When all lockers are occupied, no wristbands can be issued, so newcomers must wait (blocking). People leaving the facility return their wristbands (release).

We can observe that a mutex is a special case of a semaphore where N = 1.

Semaphores have two primary operations: acquire and release. When a semaphore has available resources, acquiring it results in holding the semaphore until it is released. When no resources are available, acquiring it results in waiting until a resource is released, at which point the semaphore is held.

We define a set of primitives: `sem_acquire()` / `sem_release()` (also written as `P()` / `V()`, `wait()` / `post()`, or `decrease()` / `increase()`):

1. A semaphore has an initial value.
2. During `acquire`, the semaphore's value decreases by 1; during `release`, it increases by 1.
3. The semaphore's value never drops below 0. If the value is 0, `acquire` will block and wait; another `release` will wake it and allow it to attempt `acquire`.

> sem_wait() decrements (locks) the semaphore pointed to by sem. If the semaphore's value is greater than zero, then the decrement proceeds, and the function returns immediately. If the semaphore currently has the value zero, then the call blocks until it becomes possible to perform the decrement (i.e., the semaphore value rises above zero).
>
> sem_post() increments (unlocks) the semaphore pointed to by sem. If the semaphore's value consequently becomes greater than zero, then another process or thread blocked in a sem_wait(3) call will be woken up and proceed to lock the semaphore.

An additional property: `acquire` (`wait`) may cause waiting (blocking), while `release` (`post`) never causes waiting (blocking).

In the final assignment, we will implement semaphores on xv6 and solve the dining philosophers problem.

PRODUCER-CONSUMER MODEL

The Producer-Consumer model is a very common model in synchronization and concurrent programming.

Producers generate data or tasks, while consumers process them. The two are separated by a buffer or queue, ensuring they do not block each other. Producers and consumers only need mutual exclusion when accessing the buffer to protect its integrity.

We can write the synchronization conditions for Producer and Consumer:

- Producer (produces data): If the buffer has space, insert data; otherwise, wait (block).
- Consumer (consumes data): If the buffer has data, remove it; otherwise, wait (block).

Between Producer and Consumer, we achieve synchronization: the production of an object must happen-before its consumption.

Typically, the buffer is a fixed-size array (no expansion after creation), such as a Ring Buffer.

Condition Variables

With synchronization conditions, we can use condition variables from the previous lesson to achieve synchronization: the synchronized object (chan) is `&buf`, protected by the spinlock `&buf.lock`. After inserting or removing data from the buffer, we use `wakeup` to notify all threads sleeping on that object.

```
struct buffer {
    spinlock_t lock;
    // ...
} buf;

bool is_full(struct buffer* b);
bool is_empty(struct buffer* b);
void put_data(struct buffer* b, void* data);
void* get_data(struct buffer* b);

void produce(void* data) {
    acquire(&buf.lock);
    while(is_full(&buf))
        sleep(&buf, &buf.lock);

    assert(!is_full(&buf));
    put_data(&buf, data);   // modify the buffer, guarded by buf.lock and cond: !full

    wakeup(&buf);
    release(&buf.lock);
}

void* consume(void* data) {
    acquire(&buf.lock);
    while(is_empty(&buf))
        sleep(&buf, &buf.lock);

    assert(!is_empty(&buf));
    void* data = get_data(&buf);    // modify the buffer, guarded by buf.lock and cond: !empty

    wakeup(&buf);
    release(&buf.lock);

    return data;
}
```

For a specific implementation, refer to the `sync_main.c` file in xv6lab10. You can try commenting out the condition checks in `producer` and `consumer` to observe whether they pass the checks.

```
        while (is_empty(&buf)) sleep(&buf, &buf.lock);
        assert(!is_empty(&buf));
```

Semaphore

We can also use semaphores to implement the producer-consumer model:

1. Producers and Consumers need to wait on two events: buffer empty or full. Thus, we create two semaphores, `avail` and `empty`, representing the current number of elements in the buffer and the number of empty slots, with initial values of `0` and `N`, respectively.
2. In `produce`, `wait(empty)` decreases the number of empty slots (`empty` --), waiting for an empty slot. After obtaining a slot, `produce` adds an object to the buffer, then `post(avail)` increases the number of elements (`avail` ++).
3. Conversely, in `consume`, `wait(avail)` decreases the number of elements (`avail` --), waiting for an element, and `post(empty)` increases the number of empty slots (`empty` ++).
4. Finally, we need mutual exclusion when accessing the buffer, so we use a `mutex` semaphore with an initial value of 1.

```
struct buffer {
    sem_t mutex;
    sem_t avail;
    sem_t empty;
    // ...
} buf;
void put_data(struct buffer* b, void* data);
void* get_data(struct buffer* b);

void produce(void* data) {
    wait(&buf.empty);   // empty space --

    wait(&buf.mutex);
    put_data(&buf, data);   // critical section for buf.
    post(&buf.mutex);

    post(&buf.avail);   // avail ++
}

void* consume(void* data) {
    wait(&buf.avail);   // avail --

    wait(&buf.mutex);
    void* data = get_data(&buf);    // critical section for buf.
    post(&buf.mutex);

    post(&buf.empty);   // empty space ++

    return data;
}
```

INVARIANT

What is an invariant? An invariant is an **assertion that must hold at any point during program execution**.

You may have learned about loop invariants in a mathematical logic course, which are conditions that always hold in a loop and can be used to prove properties of the loop.

In concurrent programming, some complex synchronization problems involve relationships between multiple shared variables. An invariant is the logical relationship between these variables. For example, if we use two `int` variables, `avail` and `empty`, to represent the number of objects and empty slots in a fixed-length Ring Buffer, the invariant is `avail + empty == N`.

If we need to temporarily break an invariant, we protect it with a Critical Section, ensuring other CPUs cannot observe the invariant being broken. As mentioned in previous lessons:

> We can package `(A, B)` into **an uninterruptible whole**. From the perspective of other CPUs, these two events occur instantaneously (i.e., atomically). Other CPUs cannot observe the intermediate state of this whole.

Typically, in a Critical Section, we access or modify multiple shared variables, but outside the Critical Section, they consistently satisfy a certain relationship.

1.2.3 Linux pthread API

The pthread (POSIX thread) library is a multithreading library for Linux. It provides the following groups of APIs:

1. Thread creation: `pthread_create`, thread joining: `pthread_join`.
2. Mutex (a type of sleeplock): `pthread_mutex_t`

```
// man pthread_mutex_init

#include <pthread.h>

pthread_mutex_t fastmutex = PTHREAD_MUTEX_INITIALIZER;

int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *mutexattr);
int pthread_mutex_destroy(pthread_mutex_t *mutex);

int pthread_mutex_lock(pthread_mutex_t *mutex);
```

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

DESCRIPTION A mutex is a MUTual EXclusion device, and is useful for protecting shared data structures from concurrent modifications, and implementing critical sections and monitors.

A mutex has two possible states: unlocked (not owned by any thread), and locked (owned by one thread). A mutex can never be owned by two different threads simultaneously. A thread attempting to lock a mutex that is already locked by another thread is suspended until the owning thread unlocks the mutex first.

1. Condition variables: `pthread_cond_t`

```
// man pthread_cond_init

#include <pthread.h>

pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

int pthread_cond_init(pthread_cond_t *cond, pthread_condattr_t *cond_attr);
int pthread_cond_destroy(pthread_cond_t *cond);

int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
```

```
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_cond_timedwait(pthread_cond_t *cond, pthread_mutex_t *mutex, const struct timespec *abstime);
```

A condition (short for ``condition variable'') is a synchronization device that allows threads to suspend execution and relinquish the processors until some predicate on shared data is satisfied. The basic operations on conditions are: signal the condition (when the predicate becomes true), and wait for the condition, suspending the thread execution until another thread signals the condition.

A condition variable must always be associated with a mutex, to avoid the race condition where a thread prepares to wait on a condition variable and another thread signals the condition just before the first thread actually waits on it.

pthread_cond_init initializes the condition variable cond, using the condition attributes specified in cond_attr, or default attributes if cond_attr is NULL. Variables of type pthread_cond_t can also be initialized statically, using the constant PTHREAD_COND_INITIALIZER.

pthread_cond_signal restarts one of the threads that are waiting on the condition variable cond. If no threads are waiting on cond, nothing happens. If several threads are waiting on cond, exactly one is restarted, but it is not specified which.

pthread_cond_broadcast restarts all the threads that are waiting on the condition variable cond. Nothing happens if no threads are waiting on cond.

pthread_cond_wait atomically unlocks the mutex (as per pthread_unlock_mutex) and waits for the condition variable cond to be signaled. The thread execution is suspended and does not consume any CPU time until the condition variable is signaled. The mutex must be locked by the calling thread on entrance to pthread_cond_wait. Before returning to the calling thread, pthread_cond_wait re acquire mutex (as per pthread_lock_mutex).

Unlocking the mutex and suspending on the condition variable is done atomically. Thus, if all threads always acquire the mutex before signaling the condition, this guarantees that the condition cannot be signaled (and thus ignored) between the time a thread locks the mutex and the time it waits on the condition variable.

1. Semaphore

```
#include <semaphore.h>

// man sem_init
int sem_init(sem_t *sem, int pshared, unsigned int value);

// man sem_post
```

```
int sem_post(sem_t *sem);

// man sem_wait
int sem_wait(sem_t *sem);
int sem_trywait(sem_t *sem);
int sem_timedwait(sem_t *restrict sem, const struct timespec *restrict abs_timeout);
```

1.2.4 Lab Exercises

1. Suppose there are two types of threads, each with multiple instances: the first type continuously prints left parentheses `(`, and the second type continuously prints right parentheses `)`. The requirement is that at any moment, the printed string is a balanced parenthesis string or its prefix, such as `()(())` or `(((`. The maximum nesting level of parentheses is limited to N.

   For example:

   - `()((()))` is a valid balanced parenthesis string with a maximum nesting depth of 3.
   - `())` is not a valid balanced parenthesis string.
   - `(((` is a prefix of `((()))` with a nesting depth of 2.

   Given a variable `count` representing the number of left parentheses exceeding right parentheses, write the synchronization conditions for these two types of threads. That is, when can each type of thread print `(` or `)`?

2. In the producer-consumer model implemented with condition variables, can the `while() sleep` loop in the `produce` method be replaced with an `if() sleep`?

   ```
   void produce(void* data) {
       acquire(&buf.lock);
       if (is_full(&buf))       // change while to if
           sleep(&buf, &buf.lock);

       assert(!is_full(&buf));
       put_data(&buf, data);   // modify the buffer, guarded by buf.lock and cond: !full

       wakeup(&buf);
       release(&buf.lock);
   }
   ```

3. Use the Linux command `man xxx` to read the manpages for `pthread_cond_signal` and `pthread_cond_broadcast`, and briefly describe the difference between them.

   Also, answer: In the implementation of the producer-consumer model, how many condition variables (`pthread_cond_t`) should we use? And when notifying a condition change (equivalent to `wakeup` in xv6), should we use `pthread_cond_signal` or `pthread_cond_broadcast`?

Built by github-actions, at 2025-05-06 11:32:50+08:00